

Potato

Potato — Semantic Word Guessing

1. Bối cảnh thực tế & Mục tiêu

Trong các hệ thống xử lý ngôn ngữ tự nhiên (**Natural Language Processing – NLP**), khả năng biểu diễn và so sánh **ngữ nghĩa của từ** là một bài toán quan trọng. Hai từ có thể khác nhau hoàn toàn về mặt ký tự nhưng lại có ý nghĩa gần nhau, chẳng hạn *hammer* và *shovel* đều liên quan đến công cụ.

Trong bài thi này, thí sinh sẽ xây dựng một hệ thống **đoán từ dựa trên độ tương đồng ngữ nghĩa**. Ban giám khảo sẽ chọn một từ bí mật từ một tập từ vựng cố định. Chương trình không được biết trước từ bí mật mà chỉ nhận được thông tin cho biết từ nào trong hai từ đang được so sánh có ý nghĩa gần với từ bí mật hơn.

Mục tiêu là tìm chính xác từ bí mật trong **không quá 30 lượt đoán**, với số lượt càng ít thì điểm càng cao.

2. Nhiệm vụ

Xây dựng một chương trình `PublicEmbeddingPlayer` có khả năng tương tác với Judge để tìm từ bí mật.

Mỗi game bắt đầu với cặp từ cố định:

`lamp` vs `potato`

Ở mỗi lượt, Judge sẽ cho biết từ nào trong cặp hiện tại gần với từ bí mật hơn. Sau đó, chương trình phải đề xuất **một từ mới** thuộc tập `vocabulary`.

Ví dụ:

```
Từ bí mật: shovel

+ {"turn": 1, "winner_word": "potato", "verdict": "second",
  "word1": "lamp", "word2": "potato"}
+ {"new_word": "rock"}

+ {"turn": 2, "winner_word": "rock", "verdict": "second",
  "word1": "potato", "word2": "rock"}
+ {"new_word": "hammer"}

+ {"turn": 3, "winner_word": "hammer", "verdict": "second",
  "word1": "rock", "word2": "hammer"}
+ {"new_word": "shovel"}

+ {"status": "win"}
```

Game kết thúc ngay khi từ được đề xuất **trùng chính xác** với từ bí mật (không phân biệt chữ hoa/chữ thường).

Mỗi từ mà chương trình đề xuất **bắt buộc phải xuất hiện trong** `dataset/vocabulary.json`.

Quy tắc tương tác

Mỗi lượt Judge gửi một JSON object có dạng:

```
{
  "turn": 1,
  "winner_word": "potato",
  "verdict": "second",
  "word1": "lamp",
  "word2": "potato"
}
```

Trong đó:

- `turn`: số thứ tự lượt, từ `1` đến `30`.
- `word1`, `word2`: hai từ đang được so sánh.
- `verdict`:
 - `first`: `word1` gần từ bí mật hơn.
 - `second`: `word2` gần từ bí mật hơn.
 - `same`: hai từ có độ gần như nhau.
- `winner_word`: từ được giữ lại để so sánh với từ đề xuất ở lượt tiếp theo.
- Nếu `verdict = "same"`, `word1` được giữ lại.

Sau mỗi lượt, chương trình trả về:

```
{
  "new_word": "rock"
}
```

3. Dữ liệu

Bộ dữ liệu cung cấp một tập từ vựng cố định gồm **1602 từ tiếng Anh**, cùng với embedding công khai tương ứng cho từng từ.

Tải về cùng với baseline tại: [đây](#)

Cấu trúc thư mục:

```
dataset/
├── vocabulary.json
└── public_embeddings.npy
```

`vocabulary.json`

Chứa **1602 từ duy nhất**, viết thường.

Từ bí mật trong game luôn thuộc tập từ này.

`public_embeddings.npy`

Ma trận embedding công khai có kích thước:

(1602, 2560)

với kiểu dữ liệu:

float32

Mỗi hàng tương ứng với một từ trong `vocabulary.json`.

Cụ thể:

`vocabulary[i]` <-> `public_embeddings[i]`

Thí sinh **chỉ được sử dụng các public embeddings được cung cấp trong dataset** để xây dựng chiến thuật dự đoán.

⚠ Lưu ý: Judge sử dụng một biểu diễn ngữ nghĩa riêng để quyết định từ nào gần từ bí mật hơn. Vì vậy, việc tối ưu chỉ dựa trên public embeddings là một bài toán quan trọng của cuộc thi.

4. Giới hạn môi trường

Chương trình được chạy trong môi trường giới hạn:

- Thời gian:** tối đa **5 phút** cho toàn bộ quá trình khởi tạo, chuẩn bị dữ liệu và chơi toàn bộ game.
- Internet:** **không có Internet**.
- Solution:** file `solution.ipynb` có kích thước tối đa **1 MB**.

Các thư viện được phép sử dụng:

```
numpy
torch
sentence-transformers
```

Tuy nhiên, trong bài thi này **không sử dụng pretrained models**. Thí sinh chỉ được sử dụng:

```
dataset/vocabulary.json
dataset/public_embeddings.npy
```

và các thư viện được phép để xây dựng thuật toán.

5. Đánh giá

Mỗi game có tối đa **30 lượt**.

Điểm của một game được tính theo lượt tìm thấy từ bí mật:

$$Score = 1.0 - 0.02 \times \max(0, t - 10)$$

trong đó t là số lượt cần để tìm ra từ bí mật.

Cụ thể:

Lượt tìm thấy		Điểm
1–10		1.00
11		0.98
12		0.96
15		0.90
20		0.80
25		0.70
30		0.60
Không tìm thấy		0.00

Điểm cuối cùng là **điểm trung bình trên toàn bộ các game**, quy đổi sang thang điểm 100:

$$FinalScore = Mean(GameScore) \times 100$$

Do đó, hệ thống không chỉ cần tìm được đáp án mà còn phải tìm được **càng sớm càng tốt**.

6. Chiến lược tham khảo

Thí sinh có thể tự do thiết kế thuật toán dựa trên public embeddings.

Một số hướng tiếp cận có thể cân nhắc:

- Nearest Neighbor:** sử dụng khoảng cách hoặc cosine similarity giữa các embedding để tìm các từ có ngữ nghĩa gần nhau.
- Candidate Search:** duy trì tập ứng viên và thu hẹp dần không gian tìm kiếm sau mỗi verdict.
- Adaptive Search:** lựa chọn từ tiếp theo dựa trên lịch sử các lần so sánh.
- Cluster-based Search:** phân nhóm các từ trong không gian embedding để tìm kiếm theo từng vùng ngữ nghĩa.
- Hybrid Strategy:** kết hợp similarity, lịch sử winner và chiến lược khám phá các vùng embedding khác nhau.
- ...

⚠ Lưu ý: Không có tập `train` được gắn nhãn. Thuật toán cần hoạt động dựa trên public embeddings và thông tin nhận được trực tiếp từ Judge trong quá trình chơi.

7. Submission Format

Thí sinh chỉ cần nộp **một file duy nhất**:

`solution.ipynb`

Notebook phải chứa phần cài đặt `PublicEmbeddingPlayer` và toàn bộ logic cần thiết để chương trình tương tác với Judge.

Không cần tạo file answer hoặc submission CSV.

Yêu cầu

- Tên file phải chính xác là `solution.ipynb`.
- Kích thước file không vượt quá **1 MB**.
- Chương trình phải đọc/ghi dữ liệu theo protocol JSON của Judge.
- Mọi `new_word` được đề xuất phải thuộc `dataset/vocabulary.json`.
- Chương trình phải xử lý tối đa 30 lượt cho mỗi game.
- `PublicEmbeddingPlayer` được khởi tạo mới ở đầu mỗi game.
- Toàn bộ các game được chạy trong cùng một lần thực thi chương trình.

8. Local Testing

Bộ dữ liệu có sẵn để thí sinh tự kiểm tra chương trình.

Có thể chạy:

```
python local_test.py solution.ipynb --limit 5
```

Local Judge sử dụng **public embeddings**, do đó điểm local chỉ mang tính chất tham khảo và **không đảm bảo phản ánh chính xác điểm trên Judge thật**.

9. Cách nộp bài

- Mở `solution.ipynb` và chỉnh sửa class `PublicEmbeddingPlayer`.
- Chạy toàn bộ notebook để kiểm tra chương trình.
- Có thể chạy local test để kiểm tra nhanh:

```
python local_test.py solution.ipynb --limit 5
```

- Lưu file `solution.ipynb`.
- Nén file `solution.ipynb` thành một file `.zip`.

Ví dụ:

```
solution.ipynb
↓
solution.zip
```

- Nộp file `solution.zip` lên hệ thống thi.

Lưu ý

- Chỉ nộp **một file .zip**.
- Bên trong file `.zip` phải chứa file `solution.ipynb`.
- Không đổi tên file notebook: tên bắt buộc là `solution.ipynb`.
- Đảm bảo file `solution.ipynb` có kích thước không vượt quá **1 MB** trước khi nén.
- Không cần tạo file answer hoặc submission CSV.

Chúc các bạn xây dựng được chiến thuật tìm từ thật nhanh và đạt điểm cao!

Submit Solution

Comments

Request clarification

Warning! Your default language, Python 3, is unavailable for this problem and has been deselected.

You have 32 submissions left

You can only submit file for this language.

Click to select a file or drag a file into this box.
Only accept ".zip". Maximum file size is 50MB.

Output Only (cat 8.32 - 9.10)

Submit!

Powered by DMOJ • AICLUB © 2026